Prueba de punta a punta: panel + API + worker + Gemini + OCI

CommunityLab · Grupo 34 · 25 de octubre de 2026 · Sergio Rebolledo (Frontend)
PR del panel: **#29** · Backend: **#28** y **#30** (ya fusionados en `main`)

1. Resumen rápido

> **Funcionó de verdad:** el flujo completo desde el panel hasta OCI. El run terminó *completado*, Gemini hizo el análisis de los mensajes, y las revisiones se guardaron en OCI con **lectura de verificación** (tanto desde el worker como desde el botón del panel).

> **No quedó probado:** la **generación de contenido con IA** (post de LinkedIn y resumen semanal). El texto que se ve en el editor es la **plantilla de respaldo**, no salida del modelo, porque la clave de Gemini es de capa gratuita y Google cortó con error 429 (cuota). Hay que repetirlo con una clave con cuota suficiente.

> **Aclaración importante:** el bucket de OCI fue un **bucket de prueba personal** (región `sa-santiago-1`), **no** el del equipo (`mx-queretaro-1`, de Michael). Ya fue borrado.

2. Qué se probó y cómo estaba armado

Pieza	Detalle
API	FastAPI (`uvicorn api.main:app`), con el backend ya fusionado (#28 + arreglo #30)
Worker	`python -m orchestration.worker`: toma el lote, analiza, genera y guarda
Panel	Streamlit (`ui/app.py`, PR #29), que solo habla con la API por HTTP
IA	Gemini vía PydanticAI, clave de capa gratuita; modelo `gemini-3.5-flash` (y luego `gemini-flash-latest`)
OCI	Bucket privado de prueba en la cuenta personal de Sergio (`sa-santiago-1`), usando el perfil de `~/.oci/config`
Datos	Lote sintético `lote_prueba_01.json` (4 mensajes: 3 válidos y 1 vacío que debe rechazarse), con un `request_id` nuevo en cada envío
Aislamiento	Directorio de datos temporal y limpio; nada tocó `data/` del repositorio ni el bucket del equipo

3. Paso a paso

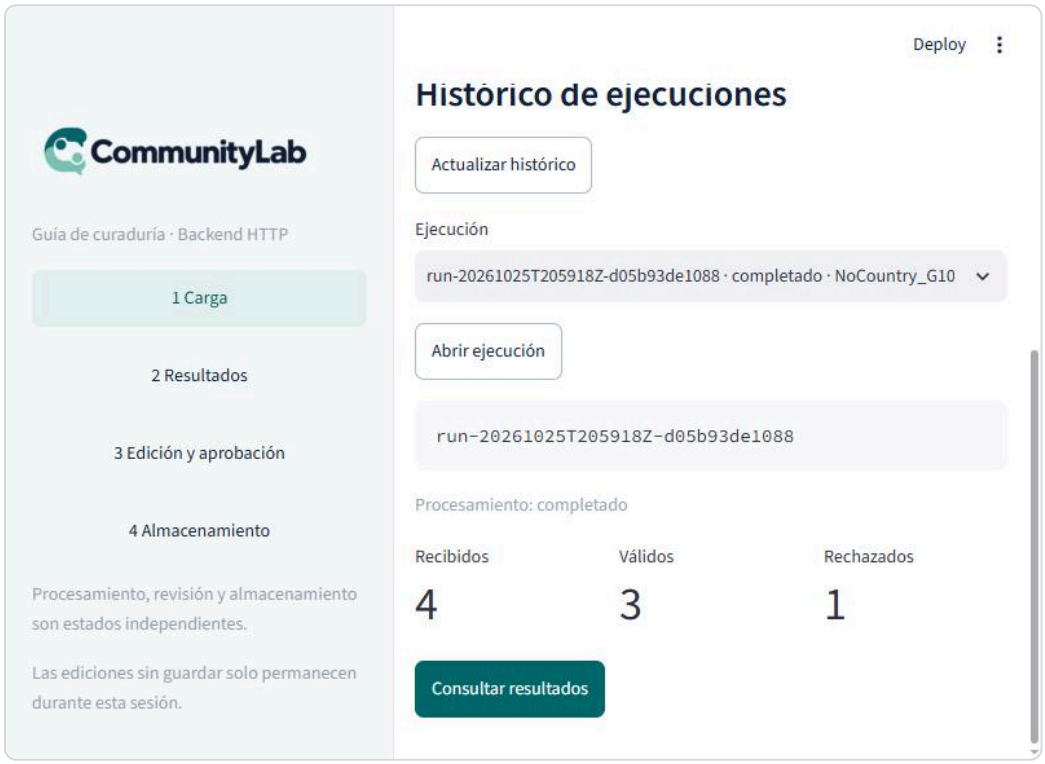
1. **Verificación previa:** `uv sync --frozen` y suite de pruebas: **374 pasan**, 1 omitida, 2 excluidas (con `main` actualizado: 377).
2. **Preparación de OCI:** se creó un bucket privado de prueba y se ajustó el `.env` para usar el perfil personal (se comentaron las variables `OCI_*` que eran placeholders).

3. **Preparación de Gemini:** clave de AI Studio puesta en `GEMINI_API_KEY` por Sergio. Se comprobó que era válida (listar modelos responde 200).
4. **Primer intento:** el run falló. El modelo por defecto del código, `gemini-2.5-flash`, ya no está disponible para claves nuevas (404). Se fijó `GEMINI_MODEL=gemini-3.5-flash`.
5. **Segundo y tercer intento:** el análisis funcionó; OCI guardó y verificó; la generación cayó a plantilla por cuota (429).
6. **Cuarto intento** con `gemini-flash-latest` y más reintentos: falló por timeouts y errores HTTP (la cuota gratuita estaba saturada). Se detuvo para no gastar más.
7. **Capturas y aprobación** sobre el run completado: recorrido por las cuatro pantallas, aprobación como revisor y guardado en OCI desde el panel.
8. **Verificación independiente:** se listó el bucket con el SDK de OCI y los tres objetos estaban ahí.
9. **Limpieza:** se borraron los objetos y el bucket de prueba.

4. Capturas

Pantalla 1 · Carga: el run recuperado desde la API

El histórico viene del backend. El run está *completado*: 4 recibidos, 3 válidos y 1 rechazado (el mensaje vacío).



Carga: run completado con 4 recibidos, 3 válidos y 1 rechazado

Pantalla 2 · Resultados: análisis hecho por Gemini

Sentimiento predominante «Mixto», temas reales (OCI, base de datos, Pydantic, modelado de datos, n8n) y categorías Logro/Duda con su puntuación. Esto **no** es la plantilla: lo calculó el modelo.



Resultados: sentimiento, temas y mensajes analizados por Gemini

Pantalla 3 · Edición y aprobación: texto de la plantilla de respaldo

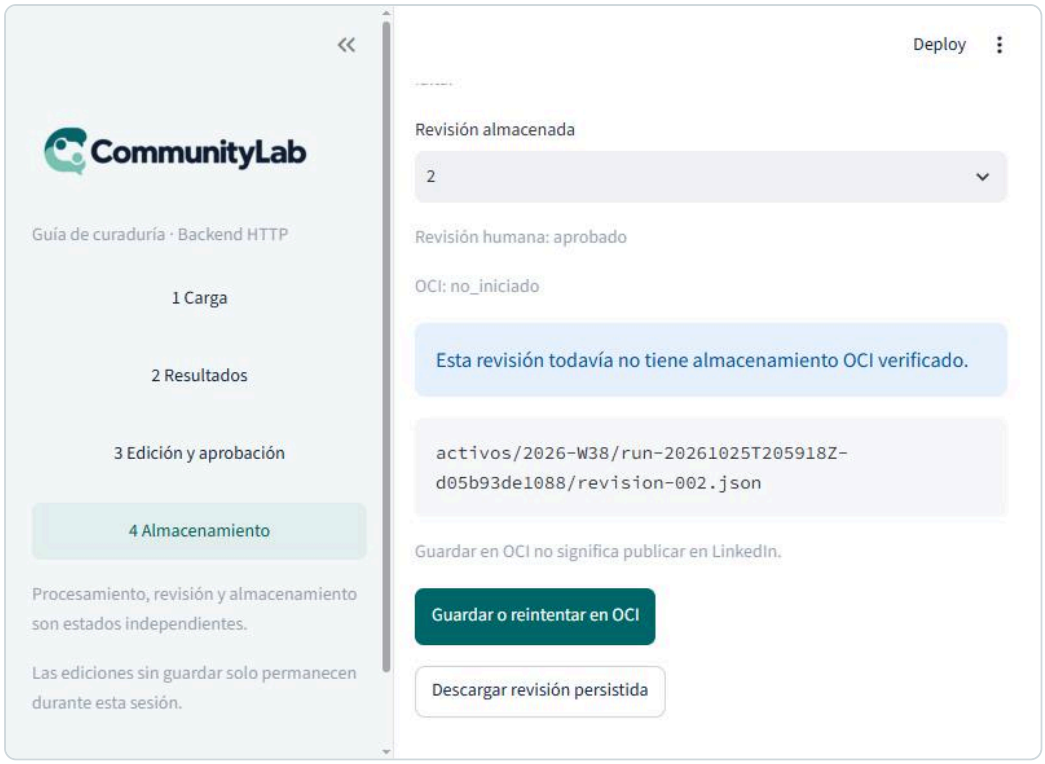
Aquí se ve el límite de la prueba: el título «Comunidad: OCI» y el cuerpo con «Contexto documentado: …» son la **plantilla de respaldo** del generador, no texto escrito por el modelo.



Editor: el contenido proviene de la plantilla de respaldo, no del modelo

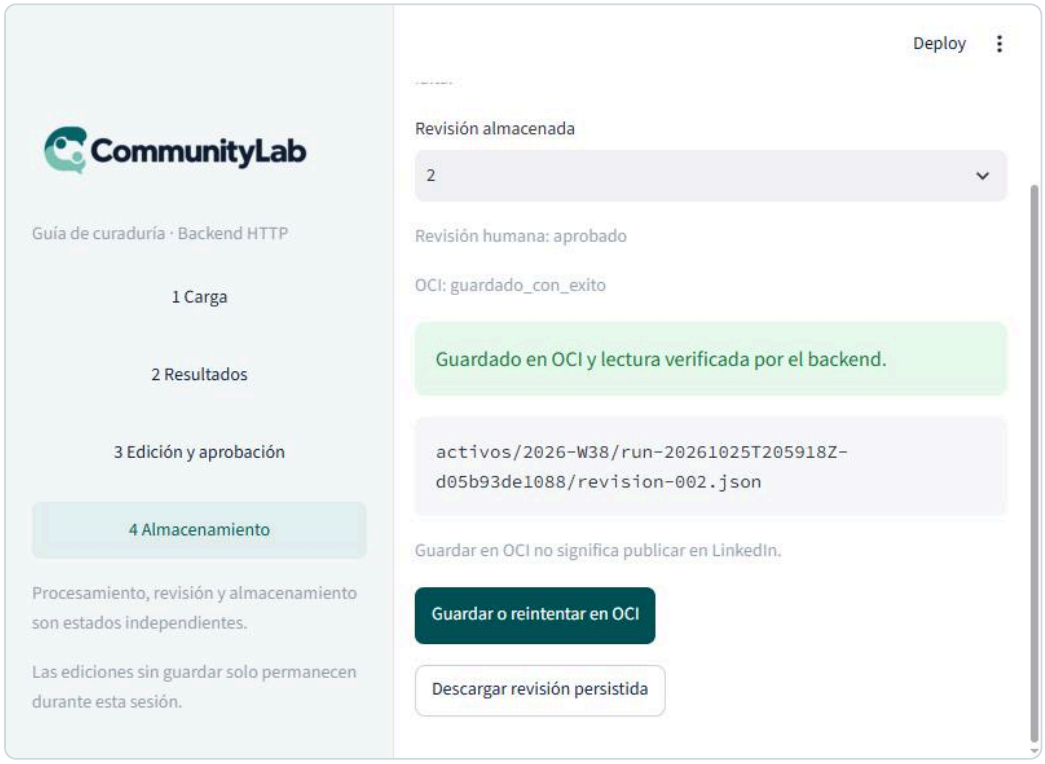
Pantalla 4 · Almacenamiento: antes y después de guardar en OCI

Antes: la revisión 2 está aprobada, pero OCI figura `no_iniciado` y el panel lo dice con honestidad: no inventa éxito.



Almacenamiento: revisión aprobada, OCI sin verificar

Después de pulsar «Guardar o reintentar en OCI»: el backend subió el objeto, lo leyó de vuelta y comparó el hash. Solo entonces el panel muestra éxito.



Almacenamiento: guardado en OCI con lectura verificada por el backend

5. Resultados

Tramo	Resultado
API → worker → pipeline	✅ Run completado (4 / 3 / 1)
Rechazo del mensaje vacío	✅ 1 registro rechazado, visible en el panel
Análisis cognitivo con Gemini	✅ Real
Generación de post y resumen con IA	❌ Cayó a plantilla de respaldo (cuota 429)
Guardado en OCI desde el worker	✅ Subida + lectura verificada
Aprobación en el panel (revisión 2)	✅ Persistida en el backend
Guardado en OCI desde el panel	✅ Lectura verificada
Objetos en el bucket	✅ Comprobado con el SDK (3 objetos)

6. Por qué falló la generación

Se midió con una ráfaga de 8 llamadas seguidas al modelo: las 6 primeras responden 200 y la séptima devuelve `429 You exceeded your current quota`. Un lote del pipeline hace más llamadas por minuto (una por mensaje para el análisis, más la generación), así que parte del trabajo supera el límite de la capa gratuita. Además `gemini-3.8-flash` respondió 503 (alta demanda) y `gemini-flash-latest` tuvo timeouts. Subir `LLM_MAX_RETRIES` a 8 no fue suficiente.

> No es un fallo del panel ni del backend: es un límite de cuota de la clave. Pero hasta repetirlo con cuota suficiente, **la generación con IA no puede darse por validada**.

7. Hallazgos para el equipo

- **Modelos Gemini retirados:** el valor por defecto del código (`gemini-2.5-flash` en `ai_modules/multiprovider_agent.py`) y el de `.env.example` (`gemini-1.5-flash`) ya no están disponibles para claves nuevas. Con una clave nueva hay que fijar `GEMINI_MODEL` . Conviene actualizar ambos.
- **El ejemplo del panel no se puede reenviar:** el botón «Enviar datos de ejemplo» usa un `request_id` fijo; tras el primer envío el backend lo trata como duplicado y devuelve el mismo run (incluso si falló). Para repetir pruebas hay que cambiar el `request_id` .
- **Revisión del PR #28:** se encontró y corrigió un 503 al arrancar la API con la base vacía (PR #30). Quedan 8 mejoras menores sin tratar; el detalle está en el comentario del #28.

8. Cómo repetirlo con el bucket del equipo

1. En `.env` : `GEMINI_API_KEY` (con cuota suficiente), `GEMINI_MODEL` y las variables `OCI_*` del equipo (las gestiona Michael). Nunca subir el `.env` al repositorio.
2. Levantar los tres procesos desde la raíz, con `COMMUNITYLAB_WEBHOOK_SECRET` definido:
   ```
   uv run --frozen uvicorn api.main:app --host 127.0.0.1 --port 8000
   ```

```
uv run --frozen python -m orchestration.worker
uv run --frozen streamlit run ui/app.py --server.address 127.0.0.1
```

3. Subir un lote desde Carga (con `request_id` nuevo), esperar el estado *completado*, revisar Resultados y Edición, aprobar y guardar en OCI.
4. Si la clave es gratuita, esperar al menos un minuto entre lotes.
5. Comprobar que el texto del editor **no** sea la plantilla de respaldo.

9. Dónde está todo

- PR del panel: github.com/No-Country-simulation/Communitylab-G10-Grupo_34-/pull/29 (revisores solicitados: Mich1971 y deymar0118-max)
- Evidencia y capturas en el repositorio: `docs/evidencias/2026-10-25-integracion-http.md` y `docs/evidencias/e2e-real/`
- Guía del panel: `ui/README.md`

Pendiente principal antes del Demo Day (27/10): repetir la generación con IA con cuota suficiente y la prueba con el bucket del equipo.